

ANASIEZE IKENNA – CLOUD & AI SOLUTIONS ENGINEER

Project: Microservices Architecture of an E-Commerce Store

Overview

I designed and deployed a scalable microservice-based e-commerce application called Mart using Docker with multiple independent services communicating through an API Gateway.

The project demonstrates container orchestration, service networking, and database integration in a distributed system.

Problem Statement

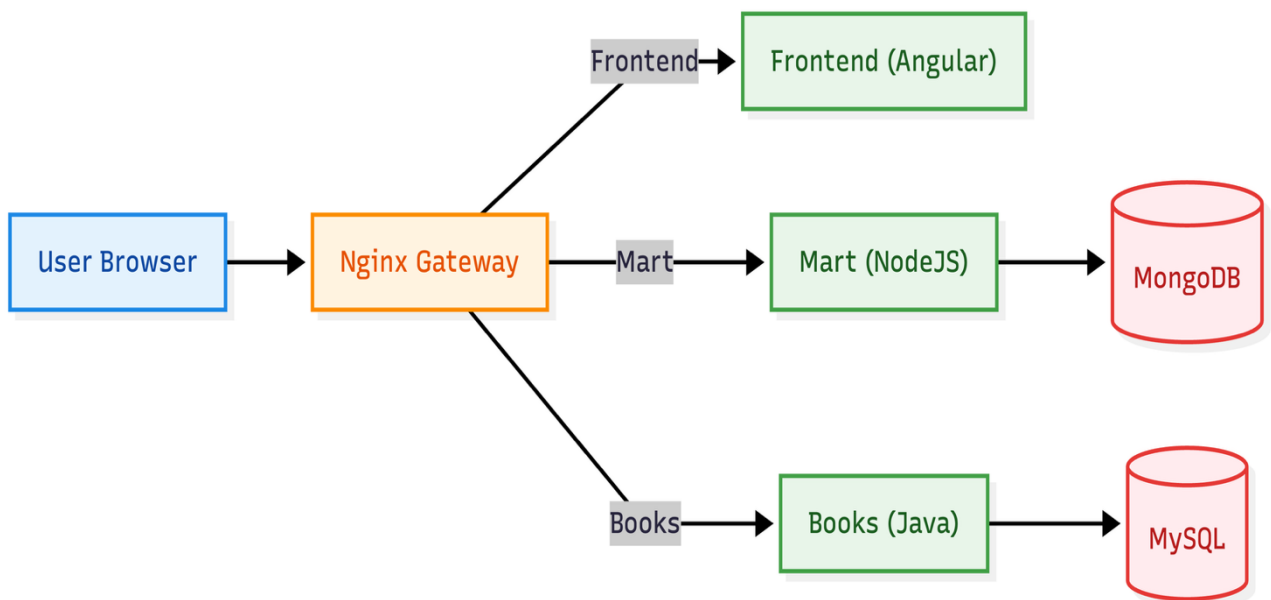
Traditional deployments do not support flexible scaling or service independence.

Modern applications require:

- + Independent service scaling
- + Separation of frontend and backend logic
- + Multiple databases per service
- + Containerized deployments for portability

Goal: Responsible for deploying and orchestrating Mart microservice architecture.

Architecture Overview



Tech Stack

- + Docker – Containerization
- + Angular – Frontend service
- + NodeJS – Mart service
- + Java – Books service
- + Nginx - API Gateway: High-performance load balancer, and web server ideal for the high traffic expected in an ecommerce scenario.
- + MongoDB (NoSQL): Excellent for flexible schema, allowing varied product catalogues with highly variable attributes (e.g., colour, size, material) that change.
- + MySQL (Relational): Best for structured data, order processing and complex queries.

Deployment Steps

- + Docker Engine and Docker Compose installed in VM
- + Built images for services using `Dockerfiles`
- + Implemented multi-containers design using `docker-compose`
- + Ran services & verified containers were running
- + Tested endpoints via browser
- + Upon access confirms database connectivity

Outcome

- + Fully functional microservice e-commerce application
- + API Gateway routing verified
- + Independent databases per service
- + Clean container lifecycle management
- + Repeatable deployment process across environment

Key Takeaway

Microservices architecture enables:

- + Service isolation
- + Independent management
- + Flexible scaling
- + Faster development iteration

Containerization ensures consistent deployment across environments.